RELATÓRIO TÉCNICO — DATA STRUCTURES AND ALGORITHMS (DSA)

Curso: Ciência da Computação (1º Ano — Turma 1CC) — FIAP
Disciplina: Data Structures and Algorithms (DSA) — Global Solution 2026.2 (*Citizen Tech*)
Projeto Integrado: Citizen Assist AI — Central do Cidadão de Aurora
Arquivo de Código-Fonte: `dsa_lista_encadeada.py`
Data de Referência e Validação: 06/11/2026

Integrantes do Grupo (Turma 1CCPZ — 1º Ano de Ciência da Computação — FIAP): - **Luiz Otávio Brito Freixo** — RM: `569977` - **Ana Beatriz Berbel Marini** — RM: `574876`

1. OBJETIVO E CONTEXTUALIZAÇÃO DO SISTEMA

No contexto da transformação digital de serviços públicos municipais (*Citizen Tech*), a **Central do Cidadão de Aurora** reúne orientações sobre emissão de documentos civis, trabalho, mobilidade urbana, direitos de atendimento prioritário e programas de transição energética residencial.

Este módulo implementa o núcleo de gerenciamento do catálogo de serviços públicos em **Python** utilizando **exclusivamente uma Estrutura de Dados Linear Dinâmica construída do zero: uma Lista Simplesmente Encadeada (`ListaEncadeadaServicos` composta por instâncias de `NoServico`)**. Em conformidade estrita com os requisitos da disciplina: 1. **Zero uso de listas nativas (`list` / `[]`):** Nenhuma estrutura nativa de vetor dinâmico do Python é empregada para armazenar, filtrar ou ordenar os registros de serviços. Todo o armazenamento e todas as transformações ocorrem por manipulação explícita de referências de memória (`cabeca` e `proximo`). 2. **Operações CRUD completas:** Inserção com validação de integridade e unicidade de chave (`id`), consulta sequencial por identificador, listagem completa de ponteiros, atualização cadastral validada e exclusão com religação segura de ponteiros em qualquer posição da lista. 3. **Algoritmos Próprios de Pesquisa e Ordenação:** Busca Sequencial Normalizada (*case-insensitive* e sem acentos) e algoritmo **Merge Sort adaptado para Lista Encadeada ($O(n \log n)$)** operando diretamente sobre os ponteiros `proximo` com divisão *Slow/Fast Pointer*.

2. MODELO DOS REGISTROS (`RegistroServico`)

Cada serviço público da Central do Cidadão de Aurora é modelado pela classe `RegistroServico`, que encapsula seis atributos obrigatórios e métodos de validação de domínio:

Atributo	Tipo	Obrigatoriedade / Regra de Validação	Papel no Domínio (Central de Aurora)
`id`	`int`	Inteiro estritamente positivo (`id > 0`) e **único** na lista	Chave primária de identificação do serviço público
`titulo`	`str`	String não vazia (`len(titulo.strip()) > 0`)	Nome oficial do serviço e referência da Ficha (`FC-01` .. `FC-07` , `ERS`)

Atributo	Tipo	Obrigatoriedade / Regra de Validação	Papel no Domínio (Central de Aurora)
`categoria`	`str`	String não vazia	Eixo temático (`Documentos` , `Trabalho e Mobilidade` , `Atendimento e Direitos` , `Energia e Sustentabilidade`)
`descricao`	`str`	String não vazia	Síntese das taxas, prazos, isenções e documentação exigida
`publico_alvo`	`str`	String não vazia	Elegibilidade, faixa etária e regras para menores/procuradores
`canal_atendimento`	`str`	String não vazia	Localização física ou plataforma digital oficial

Catálogo Inicial Carregado (`popular_dados_iniciais`)

Para permitir testes realistas de ordenação por `id` e por `titulo` , os 10 serviços oficiais da Central de Aurora são inseridos na estrutura encadeada em uma ordem inicial propositalmente embaralhada (`IDs: 7 -> 1 -> 5 -> 10 -> 3 -> 8 -> 2 -> 6 -> 9 -> 4`): - **ID 01:** Carteira de Identidade - RG: 1ª e 2ª Via (`FC-01`) — *Documentos* - **ID 02:** CPF: Inscrição, Regularização Cadastral e 2ª Via (`FC-02`) — *Documentos* - **ID 03:** Título de Eleitor: Emissão, Revisão e Transferência (`FC-03`) — *Documentos* - **ID 04:** Carteira de Trabalho Digital - CTPS (`FC-04`) — *Trabalho e Mobilidade* - **ID 05:** Certidões de Nascimento, Casamento e Óbito - 2ª Via (`FC-05`) — *Documentos* - **ID 06:** CNH: Renovação, Prazos de Validade e Exame Médico (`FC-06`) — *Trabalho e Mobilidade* - **ID 07:** Agendamento, Horários e Pagamento de Taxas (`FC-07`) — *Atendimento e Direitos* - **ID 08:** Acessibilidade, Atendimento Prioritário e Intérprete de Libras (`FC-07`) — *Atendimento e Direitos* - **ID 09:** Diagnóstico de Consumo Elétrico e Eficiência Procel/LED (`ERS-01/02`) — *Energia e Sustentabilidade* - **ID 10:** Energia Solar Fotovoltaica e Geração Compartilhada (`ERS-03`) — *Energia e Sustentabilidade*

---

3. ARQUITETURA DA LISTA SIMPLESMENTE ENCADEADA E MANIPULAÇÃO DE PONTEIROS

Diferentemente de um array contíguo em memória, a **Lista Simplesmente Encadeada** organiza seus elementos em nós independentes (`NoServico`) dispersos no *heap*, conectados de maneira lógica e sequencial por meio de referências explícitas (`proximo`).

```
flowchart LR
    Cabeca["ListaEncadeadaServicos\ncabeca (ref) | tamanho = 3"] --> N1["NoServico #1\ndado: Regis
    N1 --> N2["NoServico #2\ndado: RegistroServico(id=1)\nproximo"]
    N2 --> N3["NoServico #3\ndado: RegistroServico(id=5)\nproximo"]
    N3 --> Nulo["None\n(Fim da Lista)"]
```

3.1. Estrutura das Classes (`NoServico` e `ListaEncadeadaServicos`)

- **`NoServico` :** Contém dois atributos:
- `dado: RegistroServico` — referência para a instância que armazena os dados do serviço.
- `proximo: Optional[NoServico]` — ponteiro para o nó seguinte na sequência (ou `None` se for o último nó, denominado *cauda*).

- **`ListaEncadeadaServicos`:** Mantém o descritor da estrutura:
- `cabeca: Optional[NoServico]` — ponteiro para o primeiro nó da lista (`None` quando a lista está vazia).
- `tamanho: int` — contador atualizado em $O(1)$ a cada inserção ou exclusão bem-sucedida.

3.2. Dinâmica dos Ponteiros nas Operações Fundamentais

1. **Inserção no Fim com Verificação de Unicidade (`inserir`):** - Primeiramente, executa `registro.validar_campos()`. Caso algum campo obrigatório esteja em branco ou o `id <= 0`, a operação é abortada imediatamente em $O(1)$. - **Caso Lista Vazia (`self.cabeca is None`):** Aloca `novo_no = NoServico(registro)` e atribui `self.cabeca = novo_no`, concluindo em tempo constante $O(1)$. - **Caso Lista Não Vazia:** Um ponteiro auxiliar `atual` parte de `self.cabeca` e avança nó a nó (`atual = atual.proximo`). Em cada nó visitado, verifica se `atual.dado.id == registro.id` (rejeitando duplicatas). Ao atingir o último nó (`atual.proximo is None`), conecta `atual.proximo = novo_no` e incrementa `self.tamanho += 1`.

2. **Exclusão com Religação de Ponteiros (`excluir`):** A remoção de um nó exige reconectar a cadeia para que nenhum nó subsequente seja perdido (*memory leak* lógico) nem ocorra acesso a referência nula: - **Caso 1 — Lista Vazia (`self.cabeca is None`):** Retorna `None` imediatamente em $O(1)$ com mensagem diagnóstica segura. - **Caso 2 — Remoção na Cabeça (`self.cabeca.dado.id == id_busca`):** O ponteiro de início da lista avança para o segundo nó (`self.cabeca = self.cabeca.proximo`), isolando o antigo primeiro nó (`no_removido.proximo = None`) em tempo $O(1)$. - **Caso 3 — Remoção no Meio ou na Cauda:** Mantêm-se dois ponteiros sincronizados: `anterior` e `atual = anterior.proximo`. Quando `atual.dado.id == id_busca`, executa-se o *bypass* de ponteiro: `python anterior.proximo = atual.proximo atual.proximo = None` Se `atual` era um nó intermediário, `anterior` passa a apontar diretamente para o sucessor de `atual`. Se `atual` era a cauda (`atual.proximo is None`), `anterior.proximo` passa a valer `None`, tornando-se a nova cauda da lista.

---

4. ALGORITMOS DE BUSCA E ORDENAÇÃO IMPLEMENTADOS

4.1. Busca Sequencial por Identificador (`consultar_por_id`) e por Atributo (`pesquisar_por_atributo`)

Como listas encadeadas não possuem indexação contígua em memória (não há cálculo de endereço base $+ \text{índice} \times \text{offset}$ em $O(1)$), toda busca requer travessia sequencial de ponteiros: - **Em `consultar_por_id(id_busca)`:** Como o atributo `id` é chave única, o algoritmo interrompe a varredura assim que encontra `atual.dado.id == id_busca` (*early exit*), retornando o objeto `RegistroServico`. - **Em `pesquisar_por_atributo(termo, campo)`:** Como múltiplos serviços podem compartilhar uma mesma categoria ou substring no título, o algoritmo percorre **todos os n nós** da lista original. Tanto o termo buscado quanto o atributo do nó passam pela função `normalizar_texto()` (conversão para minúsculas e remoção de acentuação via tabela de tradução). Cada nó compatível é encadeado ao final de uma nova instância de `ListaEncadeadaServicos` usando um ponteiro `cauda_resultado` (inserção em $O(1)$ na lista de saída), mantendo 100% da execução livre de listas nativas do Python.

4.2. Ordenação Merge Sort para Lista Encadeada (`ordenar`)

Para ordenar a lista encadeada tanto por `id` (numérico crescente) quanto por `titulo` (alfabético crescente normalizado), foi implementado o algoritmo **Merge Sort Top-Down para Listas Encadeadas**, considerado o algoritmo ótimo para estruturas encadeadas por não exigir acesso aleatório por índice e por operar em $O(n \log n)$ no pior caso relinkando apenas ponteiros existentes.

O algoritmo divide-se em três rotinas encadeadas:

1. **Divisão ao Meio via *Slow/Fast Pointer* (`_dividir_ao_meio`):** - Dois ponteiros partem do início da sublista: `lento = cabeca` e `rapido = cabeca.proximo` . - Enquanto `rapido` e `rapido.proximo` não são `None` , `lento` avança **1 posição** (`lento = lento.proximo`) e `rapido` avança **2 posições** (`rapido = rapido.proximo.proximo`). - Quando `rapido` alcança o fim da sublista, `lento` encontra-se exatamente no nó central. O ponteiro `segunda_metade = lento.proximo` é salvo e a ligação central é cortada com `lento.proximo = None` , originando duas sublistas de tamanhos $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$ em tempo $O(n)$ e espaço $O(1)$.
2. **Conquista Recursiva (`_merge_sort_recursivo`):** - Aplica recursivamente a divisão até atingir sublistas unitárias (`cabeca is None or cabeca.proximo is None`), que já estão trivialmente ordenadas. A árvore de recursão possui altura exata $\lceil \log_2 n \rceil$.
3. **Intercalação de Ponteiros (*In-Place Pointer Merge* — `_intercalar_listas_ordenadas`):** - Utiliza um nó `sentinela` temporário e um ponteiro `cauda` . Compara os elementos frontais das duas sublistas ordenadas (`p_esq` e `p_dir`) por meio da função `_precede(a, b, criterio)` e aponta `cauda.proximo` para o menor nó, avançando o ponteiro correspondente. Ao final, anexa o restante da sublista não esgotada em $O(1)$ e retorna `sentinela.proximo` .

---

5. ANÁLISE FORMAL DE COMPLEXIDADE COMPUTACIONAL (NOTAÇÃO BIG-O)

Seja n o número atual de elementos (`NoServico`) armazenados na `ListaEncadeadaServicos` :

Operação / Método	Algoritmo Utilizado	Melhor Caso	Caso Médio	Pior Caso	Espaço Auxiliar	Justificativa Matemática e Estrutural
Inserção (`inserir`)	Travessia com validação de unicidade de `id` e anexação na cauda	$O(1)$	$O(n)$	$O(n)$	$O(1)$	**Melhor caso $O(1)$:** quando a lista está vazia (`cabeca is None`) ou na primeira posição se houver colisão imediata de `id` . **Caso médio e pior caso $O(n)$:** percorre todos os n nós para verificar que o `id` não está duplicado e posicionar o ponteiro no último nó antes de alocar 1 único `NoServico` .
Consulta por ID (`consultar_por_id`)	Busca Sequencial por Ponteiros com *Early Exit*	$O(1)$	$O(n)$	$O(n)$	$O(1)$	**Melhor caso $O(1)$:** o `id` buscado está no primeiro nó (`cabeca`). **Caso médio $O(n)$:** assumindo distribuição uniforme, realiza $(n+1)/2$

Operação / Método	Algoritmo Utilizado	Melhor Caso	Caso Médio	Pior Caso	Espaço Auxiliar	Justificativa Matemática e Estrutural
						comparações. **Pior caso $O(n)$:** o registro está na cauda ou não pertence à lista (n passos). Usa apenas 1 referência auxiliar `atual`.
Listagem (`listar_todos`)	Travessia Linear Completa	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Visita obrigatoriamente cada um dos n nós uma única vez seguindo `atual = atual.proximo` até `None`. Quando `n = 0` (lista vazia), executa em $O(1)$.
Atualização (`atualizar`)	Busca Sequencial por ID + Mutação Direta de Atributos	$O(1)$	$O(n)$	$O(n)$	$O(1)$	A localização do nó custa $O(1)$ no melhor caso (`cabeca`) e $O(n)$ no caso médio/pior caso. Uma vez localizado o `RegistroServico`, a substituição dos atributos em memória ocorre em $O(1)$.
Exclusão (`excluir`)	Busca Sequencial com Ponteiro Anterior + Religação	$O(1)$	$O(n)$	$O(n)$	$O(1)$	**Melhor caso $O(1)$:** remoção da `cabeca` (`self.cabeca = self.cabeca.proximo`) ou lista vazia. **Caso médio e pior caso $O(n)$:** percorre até n nós mantendo `anterior` e `atual`; o desvio de ponteiro `anterior.proximo = atual.proximo` custa $O(1)$.
Pesquisa por Atributo (`pesquisar_por_atributo`)	Busca Linear Exaustiva com Normalização de Substring	$O(n)$	$O(n)$	$O(n)$	$O(1)$ *(busca)* / $O(k)$ *(saída)*	Como vários serviços podem pertencer à mesma categoria ou conter o termo no título, o algoritmo precisa inspecionar todos os n nós em qualquer cenário ($O(n)$), alocando $k \le n$ nós na lista encadeada de retorno.
Ordenação (`ordenar`)	**Merge Sort Top-Down em Ponteiros** (*Slow/Fast Split* +	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	Em cada nível da árvore de recursão, a divisão *slow/fast* percorre $O(n)$ nós e a intercalação de ponteiros compara e religa $O(n)$

Operação / Método	Algoritmo Utilizado	Melhor Caso	Caso Médio	Pior Caso	Espaço Auxiliar	Justificativa Matemática e Estrutural
	Iterative Pointer Merge)					nós. Como a lista é bipartida ao meio a cada chamada, há $\log_2 n$ níveis: $T(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log n)$ em todos os casos. O espaço auxiliar é $O(\log n)$ referente exclusivamente aos *frames* da pilha de recursão, pois os nós são religados *in-place*.

6. EVIDÊNCIAS REAIS DE EXECUÇÃO E BATERIA DE TESTES AUTOMATIZADOS

O log abaixo foi obtido pela execução real do comando de verificação:

```
python dsa_lista_encadeada.py --test
```

Todos os casos exigidos pela especificação — incluindo **operações sobre a estrutura vazia**, **inserção de múltiplos registros**, **rejeição de ID duplicado e campos vazios**, **consulta existente e inexistente**, **atualização com estado antes e depois**, **pesquisa por título e categoria**, **ordenação Merge Sort por título e por ID** e **exclusão na cabeça, meio, cauda e inexistente** — foram validados com **zero erros**:

```
================================================================================================
  EVIDENCIAS DE TESTES AUTOMATIZADOS - DATA STRUCTURES AND ALGORITHMS (DSA)
  Estrutura: Lista Simplesmente Encadeada Propria (NoServico / ListaEncadeadaServicos)
  Universo: Central do Cidadao de Aurora (Fichas FC-01 a FC-07 + Energia Sustentavel)
================================================================================================

[TESTE 1] Operacoes sobre a Estrutura em Estado Vazio (cabeca = None, tamanho = 0)
------------------------------------------------------------------------------------------------
  -> Estado inicial: esta_vazia() = True | tamanho = 0
  -> 1.1. Tentativa de listar todos em lista vazia:
  [Lista Vazia] Nenhum servico cadastrado na estrutura encadeada (cabeca -> None).
  -> 1.2. Consultar ID 1 em lista vazia: retorno = None | msg = 'Servico ID 1 nao encontrado apos
  -> 1.3. Atualizar ID 1 em lista vazia: retorno = False | msg = 'Operacao cancelada: a lista enca
  -> 1.4. Excluir ID 1 em lista vazia  : retorno = None | msg = 'Exclusao nao realizada: a lista e
  -> 1.5. Pesquisar 'RG' em lista vazia: encontrados = 0 | msg = 'Pesquisa por 'RG' no campo 'titu
  -> 1.6. Ordenar lista vazia por ID   : retorno = True | msg = 'Lista com 0 elemento(s) ja se enc

[TESTE 2] Insercao de Multiplos Registros na Lista Encadeada (Ordem Inicial Embaralhada)
------------------------------------------------------------------------------------------------
  -> Registros inseridos via popular_dados_iniciais(): 10
  -> Estado atual: esta_vazia() = False | tamanho = 10
  -> Sequencia atual de nos e ponteiros (cabeca -> ... -> None):
  01. [ID 07] Agendamento, Horarios e Pagamento de Taxas (FC-07) | Categoria: Atendimento e Direit
  02. [ID 01] Carteira de Identidade - RG: 1a e 2a Via (FC-01) | Categoria: Documentos | Canal: Po
  03. [ID 05] Certidoes de Nascimento, Casamento e Obito - 2a Via (FC-05) | Categoria: Documentos
```

```
  04. [ID 10] Energia Solar Fotovoltaica e Geracao Compartilhada (ERS-03) | Categoria: Energia e S
  05. [ID 03] Titulo de Eleitor: Emissao, Revisao e Transferencia (FC-03) | Categoria: Documentos
  06. [ID 08] Acessibilidade, Atendimento Prioritario e Interprete de Libras (FC-07) | Categoria:
  07. [ID 02] CPF: Inscricao, Regularizacao Cadastral e 2a Via (FC-02) | Categoria: Documentos | C
  08. [ID 06] CNH: Renovacao, Prazos de Validade e Exame Medico (FC-06) | Categoria: Trabalho e Mo
  09. [ID 09] Diagnostico de Consumo Eletrico e Eficiencia Procel/LED (ERS-01/02) | Categoria: Ene
  10. [ID 04] Carteira de Trabalho Digital - CTPS (FC-04) | Categoria: Trabalho e Mobilidade | Can
  Total de registros na lista encadeada: 10

[TESTE 3] Validacao de Integridade: Rejeicao de ID Duplicado e Campos Vazios
--------------------------------------------------------------------------------------------
  -> 3.1. Inserir ID=1 ja existente : aceito = False | msg = 'Erro: Ja existe um servico cadastrad
  -> 3.2. Inserir titulo em branco  : aceito = False | msg = 'Erro de validacao: O campo obrigator
  -> 3.3. Inserir ID=-5 (negativo)  : aceito = False | msg = 'Erro de validacao: O campo 'id' deve
  -> Tamanho preservado apos rejeicoes: 10 registros.

[TESTE 4] Consulta Sequencial por ID (Existente vs. Inexistente)
--------------------------------------------------------------------------------------------
  -> 4.1. Busca por ID = 3 : Servico ID 3 localizado em 5 passo(s).
  ID do Servico     : 3
  Titulo            : Titulo de Eleitor: Emissao, Revisao e Transferencia (FC-03)
  Categoria         : Documentos
  Descricao         : Servico gratuito com emissao imediata no e-Titulo. Exige documento oficial c
  Publico-Alvo      : Obrigatorio dos 18 aos 70 anos; facultativo para jovens de 16/17 anos, 70+ e
  Canal Atendimento : Portal Titulo Net (TSE) ou Posto Eleitoral da Central de Aurora.
  -> 4.2. Busca por ID = 999 (inexistente): retorno = None | Servico ID 999 nao encontrado apos pe

[TESTE 5] Atualizacao de Registro Existente (Demonstracao Antes e Depois)
--------------------------------------------------------------------------------------------
  -> [ANTES DA ATUALIZACAO - ID 2]:
  ID do Servico     : 2
  Titulo            : CPF: Inscricao, Regularizacao Cadastral e 2a Via (FC-02)
  Categoria         : Documentos
  Descricao         : Servico gratuito e imediato (exceto regularizacao com pendencia na Receita F
  Publico-Alvo      : Qualquer pessoa fisica, inclusive recem-nascidos e estrangeiros residentes.
  Canal Atendimento : Site da Receita Federal ou Guiche de Atendimento Rapido da Central de Aurora

  -> Status da operacao atualizar(id=2): True (Servico ID 2 atualizado com sucesso.)
  -> [DEPOIS DA ATUALIZACAO - ID 2]:
  ID do Servico     : 2
  Titulo            : CPF: Inscricao, Regularizacao Cadastral e 2a Via (FC-02)
  Categoria         : Documentos
  Descricao         : Inscricao, 2a via e comprovante gratuitos na hora; regularizacao cadastral e
  Publico-Alvo      : Qualquer pessoa fisica, inclusive recem-nascidos e estrangeiros residentes.
  Canal Atendimento : Portal Receita Federal, App Gov.br e Guiche Expresso da Central de Aurora.

[TESTE 6] Pesquisa Linear por Atributo (Normalizada Case-Insensitive / Sem Acentos)
--------------------------------------------------------------------------------------------
  -> 6.1. Pesquisa por 'identidade' no campo 'titulo': 1 registro(s) encontrado(s).
  01. [ID 01] Carteira de Identidade - RG: 1a e 2a Via (FC-01) | Categoria: Documentos | Canal: Po
  Total de registros na lista encadeada: 1
```

```
  -> 6.2. Pesquisa por 'energia' no campo 'categoria': 2 registro(s) encontrado(s).
  01. [ID 10] Energia Solar Fotovoltaica e Geracao Compartilhada (ERS-03) | Categoria: Energia e S
  02. [ID 09] Diagnostico de Consumo Eletrico e Eficiencia Procel/LED (ERS-01/02) | Categoria: Ene
  Total de registros na lista encadeada: 2

  -> 6.3. Pesquisa por 'passaporte diplomatico' no campo 'ambos': 0 registro(s) encontrado(s).
  [Lista Vazia] Nenhum servico cadastrado na estrutura encadeada (cabeca -> None).

[TESTE 7] Ordenacao Merge Sort em Lista Encadeada O(n log n) - Sem Listas Nativas
------------------------------------------------------------------------------------------
  -> 7.1. Lista encadeada ordenada com sucesso por 'titulo' via Merge Sort O(n log n).
  -> Lista encadeada apos ordenacao alfabetica por TITULO:
  01. [ID 08] Acessibilidade, Atendimento Prioritario e Interprete de Libras (FC-07) | Categoria:
  02. [ID 07] Agendamento, Horarios e Pagamento de Taxas (FC-07) | Categoria: Atendimento e Direit
  03. [ID 01] Carteira de Identidade - RG: 1a e 2a Via (FC-01) | Categoria: Documentos | Canal: Po
  04. [ID 04] Carteira de Trabalho Digital - CTPS (FC-04) | Categoria: Trabalho e Mobilidade | Can
  05. [ID 05] Certidoes de Nascimento, Casamento e Obito - 2a Via (FC-05) | Categoria: Documentos
  06. [ID 06] CNH: Renovacao, Prazos de Validade e Exame Medico (FC-06) | Categoria: Trabalho e Mo
  07. [ID 02] CPF: Inscricao, Regularizacao Cadastral e 2a Via (FC-02) | Categoria: Documentos | C
  08. [ID 09] Diagnostico de Consumo Eletrico e Eficiencia Procel/LED (ERS-01/02) | Categoria: Ene
  09. [ID 10] Energia Solar Fotovoltaica e Geracao Compartilhada (ERS-03) | Categoria: Energia e S
  10. [ID 03] Titulo de Eleitor: Emissao, Revisao e Transferencia (FC-03) | Categoria: Documentos
  Total de registros na lista encadeada: 10

  -> 7.2. Lista encadeada ordenada com sucesso por 'id' via Merge Sort O(n log n).
  -> Lista encadeada apos ordenacao numerica crescente por ID:
  01. [ID 01] Carteira de Identidade - RG: 1a e 2a Via (FC-01) | Categoria: Documentos | Canal: Po
  02. [ID 02] CPF: Inscricao, Regularizacao Cadastral e 2a Via (FC-02) | Categoria: Documentos | C
  03. [ID 03] Titulo de Eleitor: Emissao, Revisao e Transferencia (FC-03) | Categoria: Documentos
  04. [ID 04] Carteira de Trabalho Digital - CTPS (FC-04) | Categoria: Trabalho e Mobilidade | Can
  05. [ID 05] Certidoes de Nascimento, Casamento e Obito - 2a Via (FC-05) | Categoria: Documentos
  06. [ID 06] CNH: Renovacao, Prazos de Validade e Exame Medico (FC-06) | Categoria: Trabalho e Mo
  07. [ID 07] Agendamento, Horarios e Pagamento de Taxas (FC-07) | Categoria: Atendimento e Direit
  08. [ID 08] Acessibilidade, Atendimento Prioritario e Interprete de Libras (FC-07) | Categoria:
  09. [ID 09] Diagnostico de Consumo Eletrico e Eficiencia Procel/LED (ERS-01/02) | Categoria: Ene
  10. [ID 10] Energia Solar Fotovoltaica e Geracao Compartilhada (ERS-03) | Categoria: Energia e S
  Total de registros na lista encadeada: 10

[TESTE 8] Exclusao de Nos e Religacao de Ponteiros (Cabeca, Meio, Cauda e Inexistente)
------------------------------------------------------------------------------------------
  -> 8.1. Remover ID 1 (Cabeca) : Servico ID 1 ('Carteira de Identidade - RG: 1a e 2a Via (FC-01)'
  -> 8.2. Remover ID 5 (Meio)   : Servico ID 5 ('Certidoes de Nascimento, Casamento e Obito - 2a V
  -> 8.3. Remover ID 10 (Cauda) : Servico ID 10 ('Energia Solar Fotovoltaica e Geracao Compartilha
  -> 8.4. Remover ID 500 (Inex.): retorno = None | Erro ao excluir: servico ID 500 nao existe na l

  -> Estado final da Lista Encadeada apos as 3 exclusoes (7 nos restantes):
  01. [ID 02] CPF: Inscricao, Regularizacao Cadastral e 2a Via (FC-02) | Categoria: Documentos | C
  02. [ID 03] Titulo de Eleitor: Emissao, Revisao e Transferencia (FC-03) | Categoria: Documentos
  03. [ID 04] Carteira de Trabalho Digital - CTPS (FC-04) | Categoria: Trabalho e Mobilidade | Can
  04. [ID 06] CNH: Renovacao, Prazos de Validade e Exame Medico (FC-06) | Categoria: Trabalho e Mo
  05. [ID 07] Agendamento, Horarios e Pagamento de Taxas (FC-07) | Categoria: Atendimento e Direit
  06. [ID 08] Acessibilidade, Atendimento Prioritario e Interprete de Libras (FC-07) | Categoria:
```

```
  07. [ID 09] Diagnostico de Consumo Eletrico e Eficiencia Procel/LED (ERS-01/02) | Categoria: Ene
  Total de registros na lista encadeada: 7
==============================================================================================
  RESULTADO FINAL DA BATERIA DE TESTES DSA: 100% APROVADO (ZERO ERROS)
==============================================================================================
```